

Multi-Class News Article Classification Using Machine Learning and Natural Language Processing: A Controlled Comparison of Ten Models on the 20 Newsgroups Corpus

Mahir Al Muntaqim
23201151

Mohammad Hasibul Amin
23201409

Fabiha Tarannum Areena
23201042

Zawad Ahsan
23201136

Department of Computer Science and Engineering, BRAC University, Dhaka, Bangladesh

Abstract

We compare ten text classification models on the 20 Newsgroups corpus under one fixed protocol: three classical models on TF-IDF features, six recurrent architectures on static word embeddings, and a fine-tuned BERT-Base encoder. Thirty tuning runs and six ablations were carried out against a single frozen set of splits, verified by a shared dataset checksum and scored by one shared evaluation module, so every number is measured the same way. BERT-Base is best at 0.7225 test macro-F1 and a uni-directional SimpleRNN is worst at 0.3405. Two findings ran against what the exploratory analysis predicted. Word2Vec covered 99.995 percent of the working vocabulary against 87.32 percent for GloVe, yet all four gated recurrent models scored higher with GloVe. And raising BERT's window from 256 to 512 tokens moved macro-F1 by 0.0016 while nearly doubling training time, so sequence length is not the binding constraint it appeared to be. Logistic Regression on TF-IDF reached 0.6852 macro-F1 in 12 seconds, beating every recurrent model here.

1 Introduction

Sorting an unstructured piece of text into one of a fixed set of topics is one of the oldest practical problems in natural language processing, and still one of the most widely deployed. News aggregators file articles into sections, discussion platforms route posts to boards, and support systems assign tickets to queues. The decision has to be made from the body of the text alone, at a volume no human reader can match.

The modelling choices span a wide range of cost, from a linear classifier that trains in seconds on a laptop to a fine-tuned transformer that needs minutes on a GPU and several gigabytes of memory. Papers introducing new architectures usually establish that the expensive option is more accurate, but the size of the gap matters just as much to anyone

choosing a model to deploy. That gap is what this study measures.

We chose the 20 Newsgroups corpus (Lang, 1995), a collection of roughly 19,000 Usenet posts from 1993 and 1994 spread across 20 discussion groups. Two properties make it a good fit. It is genuinely hard, since 20 classes is a wide label space and several groups are near neighbours in subject matter, with five sitting under the `comp` prefix and four each under `rec`, `sci` and `talk`. And because the group names are dotted paths, every misclassification can be sorted into one that stays inside a topic family and one that crosses between families, and those two kinds of error say different things about what a model has learned. The corpus is also messy. Posts carry quoted reply chains, signatures, ASCII art and hardware specification tables, so the length distribution is heavily right skewed and preprocessing becomes a real decision rather than a formality.

1.1 Research questions

The study is organised around four questions.

RQ1: how much accuracy does architectural capacity buy here, and how does the gap compare to the difference in training cost?

RQ2: does embedding vocabulary coverage predict embedding usefulness?

RQ3: do the remaining errors stay inside the correct topic family or cross between families, and does that change as models get stronger?

All ten models were trained against the same frozen splits, verified by a hash over the cleaned training text and scored by a single shared evaluation module, so macro-F1 means exactly the same thing in every row of the results table. We treat the preprocessing strategies and word representations as measured experiments rather than as pipeline decisions justified by argument alone, and we report six ablations beyond the thirty required tuning runs. Three of our results ran against the expecta-

tion formed during exploratory analysis, and we explain each rather than quietly dropping them.

2 Related Work

The corpus and sparse baselines. The 20 Newsgroups collection was introduced by [Lang \(1995\)](#) as part of a news filtering system and has been a topic classification benchmark ever since, valued for its wide label space, near balance across classes, and hierarchical labels that make error analysis interpretable ([Yin et al., 2019](#); [Tunstall et al., 2022](#)). The dominant approach for two decades was a term-weighted bag of words fed to a linear classifier. [Spärck Jones \(1972\)](#) introduced inverse document frequency and [Salton and Buckley \(1988\)](#) established the weighting schemes still in use. [Joachims \(1998\)](#) argued that text categorisation suits linear models because the feature space is high dimensional and most features are relevant, and [McCallum and Nigam \(1998\)](#) established the multinomial event model as the standard naive Bayes variant for text. [Breiman \(2001\)](#) introduced random forests, which are used far less for sparse text because axis-aligned splits on a high-dimensional sparse matrix use each tree inefficiently, a pattern our results reproduce.

Embeddings and recurrence. [Mikolov et al. \(2013\)](#) introduced skip-gram with negative sampling, which can be trained on a target corpus directly, while [Pennington et al. \(2014\)](#) factorised a global co-occurrence matrix built from very large general corpora. The trade-off between domain fit and estimation quality is what RQ2 asks about, and it is more often assumed than measured. On the architecture side, the simple recurrent network of [Elman \(1990\)](#) applies one transformation per timestep, which makes gradients over long sequences unstable ([Pascanu et al., 2013](#)). [Hochreiter and Schmidhuber \(1997\)](#) solved this with the LSTM cell, [Cho et al. \(2014\)](#) proposed the GRU as a lighter two-gate variant, and [Schuster and Paliwal \(1997\)](#) introduced bidirectional recurrence. These four designs form our recurrent tier. [Yang et al. \(2016\)](#) and [Kim \(2014\)](#) showed that attention pooling and convolution also help, but both fall outside the toolkit fixed for this project.

Transformers and preprocessing. [Vaswani et al. \(2017\)](#) replaced recurrence with self-attention, so the path between any two tokens is a single operation regardless of distance, and [Devlin et al. \(2019\)](#)

pretrained a bidirectional encoder that transfers to classification by fine-tuning. [Sun et al. \(2019\)](#) recommended learning rates between $2e-5$ and $5e-5$, which is the grid we adopt, and BERT uses WordPiece subword units ([Sennrich et al., 2016](#)). On preprocessing, [Uysal and Gunal \(2014\)](#) showed that such choices interact with the domain and the classifier rather than helping uniformly, which is why we treat them as an ablation.

3 Methodology

3.1 Dataset

We use the Hugging Face distribution of the corpus, `SetFit/20_newsgroups`, loaded through the `datasets` library ([Lhoest et al., 2021](#)). It ships 11,314 training and 7,532 test posts, 18,846 in total, each with a raw text body, an integer label and a label name. The split boundary in the published benchmark is temporal, with training posts preceding test posts in time, and we preserve it exactly. Reshuffling the two halves would leak later posts into training and inflate every score reported here. The 20 labels collapse into 7 families by prefix, with 5 classes under `comp`, 4 each under `rec`, `sci` and `talk`, and one each under `alt`, `misc` and `soc`. We use this grouping throughout the error analysis.

One property of this distribution matters for the transformer tier. The headers, footers and quoted reply blocks that dominate the raw Usenet archive have already been stripped. We measured what survives in the training split: 1.1 percent of documents still contain a `From:` line, 0.8 percent contain `Subject:`, and none contain quoted reply markers. Metadata leakage is the standard objection to results on this corpus, since a model can learn a signature block instead of a topic, and at these rates it is not a live concern here.

3.1.1 Class balance and length

Figure 1 shows the per-class counts by split. The corpus is close to balanced, with between 628 and 999 documents per class and an imbalance ratio of 1.59 between `rec.sport.hockey` and `talk.religion.misc`, so no resampling or class weighting is applied. We nonetheless select every configuration on macro-F1 rather than accuracy, because the three smallest classes turn out to be the ones every model confuses most and macro-F1 weights each class equally.

Figure 2 shows the length distribution, which

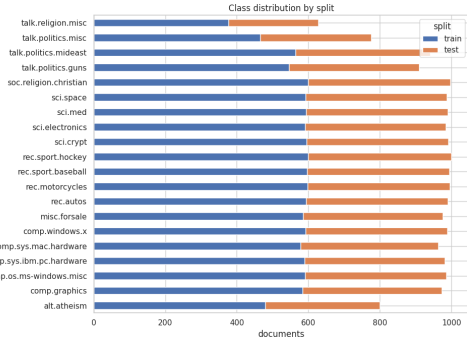


Figure 1: Documents per class, stacked by split. The corpus is close to balanced, with an imbalance ratio of 1.59.

is heavily right skewed. Measured on the cleaned text, the median is 83 words while the mean is 185.8 on train and 175.3 on test, and the longest cleaned document runs to 11,765 words. The gap comes from quoted reply chains, where an author quotes an entire earlier post before adding two lines. With the sequence length capped at 250 tokens, 85.5 percent of documents fit entirely inside the window; the 90th percentile sits at 331 words and the 99th at 1,751. The skew also means information in a long document must survive a long chain of recurrent updates to reach the classifier, which is the mechanism RQ1 probes.

3.1.2 Data quality

Four issues appear in the raw corpus. There are no missing values, but 515 documents are empty strings and 557 fall below 10 characters after cleaning, there are 559 exact duplicate bodies, and 226 test documents appear verbatim in training.

We handle these asymmetrically. Empty and near-empty documents are removed from both splits, since a document with no surviving tokens contributes nothing but a guaranteed error. Exact duplicates are dropped from training only, so a repeated post cannot be memorised, but they are left in test because the test set is the benchmark and should not be curated to a model’s advantage. The 226 overlapping documents are reported rather than deleted, since they are a property of the published benchmark and removing them would produce a test set that no other paper measures against. After these steps training holds 10,882 documents and test holds 7,262.

3.1.3 Vocabulary by family

Figure 3 shows the most frequent terms per family after stopword removal. The families separate

	light	full
Vocabulary size	66,111	54,253
Total tokens	2,221,923	1,062,081
Mean tokens per doc	196.4	93.9
Terms with frequency 1	30,109	26,374
<i>Embedding coverage of the 20,000-word vocabulary</i>		
Word2Vec	19,998 / 19,999	(99.995%)
GloVe	17,464 / 19,999	(87.32%)

Table 1: Top: effect of the two preprocessing strategies on the training split. Bottom: coverage of the working vocabulary by each embedding source. Every miss becomes a zero row, which the model reads as a word with no meaning.

on obvious content words, but two observations matter later. The `comp` cloud is dominated by `ax` and `max`, which come from a single long post of encoded binary data, an example of the noise the corpus carries. And the `alt` and `soc` clouds share their strongest term, `god`, an early warning that the religion-related classes will be hard to separate on vocabulary alone.

3.2 Preprocessing

We implemented two strategies and kept the untouched raw text alongside both.

The **light** strategy lowercases the text, strips email addresses and URLs, removes every character that is not a lowercase letter or whitespace, and collapses repeated whitespace. The character filter deletes digits and punctuation in one pass. This is a deliberate trade-off. It removes model numbers and version strings that could distinguish hardware groups, but it also removes the arithmetic, alignment characters and quoting markers that make up much of the corpus noise.

The **full** strategy adds stopword removal, drops tokens of two characters or fewer, and lemmatises the survivors with the WordNet lemmatiser (Miller, 1995; Bird et al., 2009). The lemmatiser is called twice, first with its default noun assumption and then with an explicit verb tag. This matters more than it looks: a single default call maps `drives` to `drive` but leaves `driving` unchanged, so two inflections stay separate features, while chaining the verb pass collapses all three onto one form. We prefer lemmatisation to stemming (Porter, 1980) because stemmed forms are not readable, and the per-class top-term inspection in Section 3.4 depends on being able to read the features.

Table 1 shows what the two strategies produce on the training split.

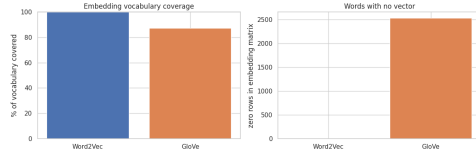


Figure 4: Coverage of the 20,000-word working vocabulary by each embedding source. The 2,535 words GloVe misses become zero rows in the embedding matrix.

lary: `sci.crypt` is led by `key`, `encryption`, `chip` and `clipper`, and `comp.windows.x` by `window`, `server`, `motif` and `widget`.

Word2Vec. A skip-gram model trained on the 9,249 cleaned training documents with 50 dimensions, a window of 5, a minimum count of 2 and 10 epochs, using `gensim` (Rehurek and Sojka, 2010), giving a vocabulary of 25,349 words. Nearest-neighbour probes are sensible: `car` retrieves `bike`, `honda`, `tire` and `truck`, and `encryption` retrieves `nsa`, `cryptography` and `rsa`.

GloVe. The pretrained `glove.6B.50d` vectors, 400,000 words estimated from six billion tokens of Wikipedia and Gigaword.

Both embedding sources were aligned to the same Keras tokenizer index so that the S1 and S2 configurations in Section 3.5 differ only in the matrix loaded. Row zero is left as zeros for padding. Figure 4 and the lower half of Table 1 show the coverage gap this produces.

The natural prediction from those numbers is that Word2Vec should win, because 2,535 zero rows is a large hole. Section 4.5 reports what actually happened.

WordPiece. BERT uses its own pretrained WordPiece vocabulary of 30,522 subword units on the raw text, capped at 256 tokens. Because unseen words decompose into known pieces, coverage is not a variable for this tier. Subword splitting is aggressive on this corpus: the raw text expands at 1.88 WordPiece tokens per word, because URLs, code fragments and encoded binary blocks have no vocabulary entries and shatter into many pieces.

3.5 Model architectures

Ten models were trained across three tiers, with three configurations each, for thirty required runs. Table 6 in the appendix lists every configuration.

Classical tier. Logistic Regression, Multinomial Naive Bayes and Random Forest (Pedregosa et al.,

2011), all on the shared TF-IDF matrix. Each varies the parameter controlling its own capacity: inverse regularisation strength, Laplace smoothing, and ensemble size with feature subsampling. Logistic Regression is capped at 2,000 iterations to guarantee convergence.

Recurrent tier. Six models built from one shared factory in Keras (Chollet et al., 2015; Abadi et al., 2016): SimpleRNN, LSTM and GRU, each with and without a bidirectional wrapper. The architecture is an embedding layer initialised from the chosen matrix, one recurrent layer returning only its final state, a dropout layer (Srivastava et al., 2014), and a softmax over 20 classes. In the bidirectional case a second pass reads the reversed sequence and the two final states are concatenated.

The embedding matrix dominates the parameter count. It holds exactly one million weights for every model, while the recurrent and output layers together range from 8,660 trainable weights for a 64-unit SimpleRNN to 61,460 for a 64-unit bidirectional LSTM. In the S1 and S2 configurations the embedding is frozen, so only that small tail is learned. In S3 the full million-parameter matrix updates, which is why S3 wins for all six architectures.

Two implementation details matter. Masking is enabled on the embedding layer so padding positions do not update the hidden state after a document ends: with a median length of 83 tokens in a 250-token window, most positions in most examples are padding, and without masking the final hidden state would be dominated by them. Recurrent dropout is left at zero because any non-zero value disables the optimised GPU kernel for no measured benefit. All recurrent runs use Adam (Kingma and Ba, 2015) at 1e-3 with gradient clipping at norm 1.0, batch size 64, and early stopping on validation accuracy with patience 5 and best-weight restoration, capped at 30 epochs.

Transformer tier. `bert-base-uncased` with a fresh classification head, fine-tuned end to end through the `transformers` Trainer (Wolf et al., 2020) at 256 tokens on raw text. Weight decay is 0.01, warmup is 10 percent of total steps, gradients are clipped at norm 1.0, and mixed precision is enabled. Evaluation runs each epoch and the best checkpoint by validation macro-F1 is restored at the end, which is the same selection rule the recurrent models use.

4 Results

4.1 Consolidated comparison

Table 2 gives the test set result for every model at its best configuration.

The spread from best to worst is 0.3820 macro-F1. Accuracy and macro-F1 track each other closely, which is expected on a near-balanced corpus, and where they separate macro-F1 is always the lower of the two, meaning the model is weaker on the smaller classes than accuracy alone suggests. The gap is widest for the two SimpleRNN variants and narrowest for the transformer at 0.0120.

The ordering is not by tier. Logistic Regression and Naive Bayes, both on plain TF-IDF, sit above all six recurrent models, and only the transformer beats them, by 0.0373 macro-F1 over the best classical model. Random Forest is the exception within its tier, finishing eighth: axis-aligned splits on a 20,000-dimensional sparse matrix mean most trees see mostly zeros, and the model has to build the same decision boundary that a linear model expresses in one weight vector.

4.2 Accuracy against training cost

Figure 6 in the appendix plots macro-F1 against training time on a log scale, because the times span five orders of magnitude.

This is the answer to RQ1. BERT reaches 0.7225 in 602 seconds on a GPU. Logistic Regression reaches 0.6852 in 12 seconds on a CPU, and Naive Bayes reaches 0.6843 in 0.02 seconds. The transformer buys 0.0373 macro-F1 for roughly fifty times the training time of the best classical model, and roughly thirty thousand times the time of the second best. The recurrent tier is the poor trade in this picture: it costs more than the classical models and delivers less.

4.3 Confusion structure

Figure 5 shows the row-normalised confusion matrices for the best and worst models at full size. Row normalisation matters here, since a bright off-diagonal cell then means a class is losing a large share of its own documents rather than simply being a large class.

Both matrices show block structure along the diagonal, corresponding to the label families. The SimpleRNN matrix additionally shows vertical stripes at `soc.religion.christian` and `comp.sys.ibm.pc.hardware`, where several unrelated true classes collapse onto one predic-

tion. That is the signature of a model that has not learned to discriminate and falls back on a handful of frequent outputs.

Figure 7 in the appendix zooms into the five `comp` classes for the strongest model of each tier. BERT keeps 0.727 of each class on average inside this block against 0.673 for Logistic Regression and 0.641 for Bi-LSTM. This is where the transformer earns its cost, because the `comp` classes are separated by what a word means in context: `driver`, `window` and `bus` appear in all five groups and only the surrounding sentence resolves which one is meant. Self-attention supplies exactly that.

The three religion-related classes reverse the ordering (Figure 8). Logistic Regression reaches an average diagonal of 0.519, slightly ahead of BERT at 0.511 and well ahead of Bi-LSTM at 0.469. These classes are not separated by word sense. They discuss the same subject from different positions using largely the same vocabulary, so the discriminating signal is which specific terms appear at all rather than what they mean. TF-IDF measures precisely that. The largest single confusion in the BERT matrix is `talk.politics.misc` predicted as `talk.politics.guns`, taking 32.1 percent of that class, followed by three confusions among the religion trio at 16.9, 19.7 and 23.9 percent.

4.4 Where the errors fall

Table 5 in the appendix splits every model’s errors into those that stay inside a label family and those that cross between families.

This answers RQ4. If errors were spread at random across the other 19 labels, roughly 15 percent would land in the same family by chance. Every model runs between 34.5 and 46.9 percent, so the errors follow the semantics of the label set rather than noise, even for the model that gets two thirds of the corpus wrong.

The share is highest for the two weakest models, which looks backwards until the absolute counts are read alongside it. SimpleRNN makes 1,950 within-family errors and BERT makes 770. The weak models pick up coarse family-level vocabulary and then fail on the fine distinctions inside a family, so a large share of their many errors lands in the right neighbourhood. The strong models fix most of those, and what survives is a smaller set concentrated in genuinely confusable pairs. One caveat on the metric: the prefix taxonomy puts

#	Model	Cfg	Tier	Representation	Test acc.	Test macro-F1	Val macro-F1	Time (s)
1	BERT-Base	B3	transformer	WordPiece	0.7345	0.7225	0.7753	602.40
2	LogisticRegression	LR-C2	classical	TF-IDF	0.6982	0.6852	0.7377	12.11
3	MultinomialNB	NB-C2	classical	TF-IDF	0.7038	0.6843	0.7348	0.02
4	Bi-LSTM	S3	recurrent	Word2Vec	0.6767	0.6627	0.7283	104.60
5	LSTM	S3	recurrent	Word2Vec	0.6720	0.6551	0.7122	78.90
6	Bi-GRU	S3	recurrent	Word2Vec	0.6409	0.6304	0.7069	89.90
7	GRU	S3	recurrent	Word2Vec	0.6304	0.6214	0.6888	51.70
8	RandomForest	RF-C2	classical	TF-IDF	0.6373	0.6134	0.6593	47.06
9	BiSimpleRNN	S3	recurrent	Word2Vec	0.4683	0.4328	0.4640	190.03
10	SimpleRNN	S3	recurrent	Word2Vec	0.3912	0.3405	0.3744	88.31

Table 2: Test set performance for all ten models at their best configuration, ranked by macro-F1. Configuration selection was made on validation macro-F1; the test set was evaluated once per model afterwards.

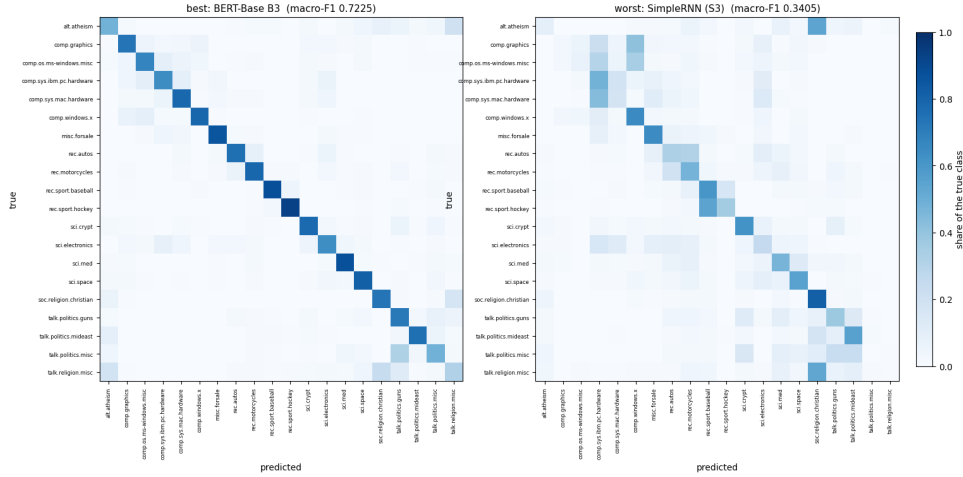


Figure 5: Row-normalised confusion matrices for the best model (left) and worst model (right). Both show block structure along the label families, but SimpleRNN also shows vertical stripes where several classes collapse onto a small number of frequent predictions.

the three religion-related classes in three different families, so confusions among them count as cross-family failures when in substance they are not.

4.5 The embedding contrast

Configurations S1 and S2 differ only in which embedding matrix is loaded. Units, dropout, learning rate, batch size and the frozen flag are all identical, so any difference is attributable to the embedding source alone. This comparison is repeated across all six recurrent architectures, and the left panel of Table 3 reports it.

This is the answer to RQ2, and it is not the answer the coverage numbers predicted. The result splits cleanly by architecture: both ungated models prefer Word2Vec, by 0.0498 and 0.0234, while all four gated models prefer GloVe, by margins from 0.0026 to 0.0247.

Coverage and quality are different properties, and the split shows which one each architecture depends on. GloVe was estimated from six billion tokens of general English, so the 87.32 percent of

our vocabulary it does cover sits in a well-shaped space with meaningful geometry. Our Word2Vec was trained on about 9,000 documents, so it has a vector for nearly every word but each vector rests on very little evidence. A model that cannot build context has to decide from individual token identities, and for it a noisy vector still beats a zero vector, so coverage wins. A model that can build context exploits the finer structure in better-estimated vectors and loses less from a missing token because the surrounding words supply the meaning, so quality wins.

4.6 Architecture and sequence length

The right panel of Table 3 orders the recurrent tier by how much gating each architecture has.

The single largest architectural gain anywhere in this study is the step from SimpleRNN to GRU, worth 0.2809 macro-F1. That step adds gating and nothing else, and it works because gating lets part of the hidden state pass forward unchanged instead of being rewritten by a matrix multiplication at

Model	GloVe	Word2Vec	Delta
SimpleRNN	0.2853	0.3351	+0.0498
BiSimpleRNN	0.3880	0.4114	+0.0234
GRU	0.6287	0.6131	-0.0156
Bi-GRU	0.6270	0.6104	-0.0166
LSTM	0.6120	0.6094	-0.0026
Bi-LSTM	0.6378	0.6130	-0.0247

(a) Validation macro-F1, S1 (GloVe) against S2 (Word2Vec). Delta is Word2Vec minus GloVe.

Model	Test acc.	Macro-F1	Time (s)
SimpleRNN	0.3912	0.3405	88.3
BiSimpleRNN	0.4683	0.4328	190.0
GRU	0.6304	0.6214	51.7
Bi-GRU	0.6409	0.6304	89.9
LSTM	0.6720	0.6551	78.9
Bi-LSTM	0.6767	0.6627	104.6

(b) The recurrent tier at S3, ordered by amount of gating.

Table 3: Two controlled contrasts inside the recurrent tier. Left: the embedding source, with everything else held fixed. Both ungated models prefer Word2Vec and all four gated models prefer GloVe. Right: the architecture, with the step from SimpleRNN to GRU worth 0.2809 test macro-F1.

Model	clean	light	Delta
LogReg	0.7377	0.7218	+0.0159
NB	0.7348	0.7179	+0.0169
RandomForest	0.6593	0.6518	+0.0075
LSTM	0.6219	0.5889	+0.0330
256 tok. 512 tok.			
BERT (test)	0.7225	0.7241	+0.0016

Table 4: The six ablations. The first four rows are validation macro-F1 under the two preprocessing strategies; the last is test macro-F1 under two BERT sequence lengths.

every timestep. Given the length distribution in Figure 2, that is exactly the property this corpus demands.

Adding a bidirectional pass gives much smaller gains: 0.0091 for the GRU pair and 0.0077 for the LSTM pair. The exception is instructive. For SimpleRNN the bidirectional gain is 0.0923, ten times the gated case. A model that cannot retain early tokens benefits from a second pass reaching them from the other end. A model that can retain them already has that information and gains nothing.

4.7 Ablations

Six runs were carried out beyond the thirty required. Table 4 collects them.

Preprocessing. This answers RQ3. The cleaned strategy wins on all four models tested, but the margin is modest on the classical tier, between 0.0075 and 0.0169 validation macro-F1, and larger on the LSTM at 0.0330. The mechanisms differ, which is why the sizes differ. For TF-IDF the cost of keeping stopwords is feature slots under a 20,000-term cap. For a sequence model it is timesteps inside a 250-token window, and since the light strategy leaves 196.4 tokens per document against 93.9 for the full strategy, a light-preprocessed document

uses roughly twice as much of the window on the same content. The honest reading of the classical numbers is that the preprocessing choice matters less than the cap it interacts with.

Sequence length. Raising BERT’s window from 256 to 512 tokens cuts truncation from 24.4 percent of documents to 9.1 percent, so the model sees 15.2 percent more of the corpus. That bought 0.0016 macro-F1 for 1.85 times the training time. The small bidirectional gains point the same way: if identifying vocabulary appeared late in these documents, both reading backwards and reading further forward would help, and neither does much. Topic signal in Usenet posts is front-loaded, which makes sense given that the first paragraph usually states the question.

4.8 Best and worst configurations

Within each model the tuning pattern is consistent. Logistic Regression peaked at $C = 5.0$ rather than at either end of its range, so both under- and over-regularisation cost accuracy. Naive Bayes peaked at $\alpha = 0.1$: with 20,000 features and about 460 documents per class, heavy smoothing washes out rare but discriminating terms while almost no smoothing overfits to them. Random Forest was worst when feature subsampling dropped to 2 percent, which starved each split of usable features.

Across the recurrent tier S3 wins for all six architectures. It is the only configuration that unfreezes the embedding matrix, and letting a million embedding parameters adapt to the corpus is worth between 0.04 and 0.12 validation macro-F1 over the frozen Word2Vec baseline. The size of that gain grows with the amount of gating: 0.039 and 0.053 for the two SimpleRNN variants, 0.076 and 0.097 for the GRU pair, and 0.103 and 0.115 for the LSTM pair. A model that can carry information

across the sequence has more use for a better-fitted embedding than one that cannot. For BERT, the highest learning rate with the larger batch and one extra epoch was best, consistent with Sun et al. (2019).

One caveat applies to all of these numbers. Nine of the twenty recurrent runs used their full 30-epoch budget without early stopping ever firing, and all three BERT configurations peaked at their final epoch, so every one of those runs was still improving when training stopped and their scores are floors. We report this rather than correcting it, because raising the budget now would mean retraining every model in the tier to keep the comparison fair.

5 Conclusion

We trained ten models across three architectural families on the 20 Newsgroups corpus, using thirty tuning runs and six ablations, all against frozen shared artifacts verified by a single dataset checksum and scored by one shared evaluation module.

Takeaways. The ranking follows each architecture’s capacity to carry information across a long document. Ungated recurrence performs worst at 0.3405 macro-F1, gating closes most of the gap at 0.6627, and attention with pretraining performs best at 0.7225. But a simple classical model on TF-IDF stays competitive: Logistic Regression at 0.6852 and Naive Bayes at 0.6843 both beat every recurrent model here, in 12 seconds and 0.02 seconds respectively. Newsgroup topics are strongly keyword-driven, which suits a bag-of-features representation, and this is why the transformer’s advantage is much smaller than its compute cost implies. What BERT actually buys is the `comp` block, where word sense in context is the discriminating signal. Three controlled comparisons contradicted the expectation formed during exploratory analysis, and we report all three as measured: embedding coverage did not predict embedding usefulness, sequence length was not the binding constraint, and the transformer lost the religion block to TF-IDF.

Limitations. The embedding dimension was fixed at 50 for both sources, the smallest GloVe release available. Holding it constant keeps the S1 against S2 contrast clean, but 50 dimensions is a low ceiling for 20 classes and is probably the largest single constraint on the recurrent results. Several configurations used their entire epoch bud-

get while still improving, so those numbers are floors. The test split holds 7,262 rows rather than the standard 7,532 because of our quality filtering, so these figures are not directly comparable to published results without saying so. Validation scores exceed test scores for every model, which follows from the temporal split. This affects all ten models equally so the ranking holds, but absolute test numbers sit below what validation suggests. The two tiers also do not read the same span of each document, since 250 tokens of cleaned text covers more of an original post than 256 WordPiece tokens of raw text. Finally, TF-IDF fed only the classical tier and the embeddings fed only the recurrent tier, so the representation comparison is confounded with the architecture comparison.

Future work. Raise the embedding dimension to 100 or 300 and re-run the recurrent tier. Extend the epoch budget for the configurations that exhausted theirs, so every model is compared at convergence. Run one classical model on averaged embedding vectors and one recurrent model on TF-IDF, which would separate the representation effect from the architecture effect that is currently confounded. And merge the three religion-adjacent classes and re-measure, since Logistic Regression already matches BERT on that block, which suggests the remaining error there is a property of a label set that separates classes by stance rather than by topic.

Acknowledgements

This work was completed as the term project for CSE440. All experiments ran on Google Colab. Mahir Al Muntaqim built the dataset pipeline, exploratory analysis, preprocessing, splits and all three word representations. Mohammad Hasibul Amin trained the three classical models and the two SimpleRNN variants. Fabiha Tarannum Areena trained the LSTM and bidirectional LSTM models and wrote the shared evaluation module. Zawad Ahsan trained the GRU, bidirectional GRU and BERT models and assembled the consolidated results.

References

Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul

- Tucker, Vijay Vasudevan, Pete Warden, and 3 others. 2016. TensorFlow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation*, pages 265–283.
- Steven Bird, Ewan Klein, and Edward Loper. 2009. *Natural Language Processing with Python*. O’Reilly Media.
- Leo Breiman. 2001. Random forests. *Machine Learning*, 45(1):5–32.
- Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1724–1734.
- François Chollet et al. 2015. Keras. <https://keras.io>.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186.
- Jeffrey L. Elman. 1990. Finding structure in time. *Cognitive Science*, 14(2):179–211.
- Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural Computation*, 9(8):1735–1780.
- Thorsten Joachims. 1998. Text categorization with support vector machines: Learning with many relevant features. In *Proceedings of the 10th European Conference on Machine Learning*, pages 137–142. Springer.
- Yoon Kim. 2014. Convolutional neural networks for sentence classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1746–1751.
- Diederik P. Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations*.
- Ken Lang. 1995. NewsWeeder: Learning to filter netnews. In *Machine Learning Proceedings 1995*, pages 331–339. Morgan Kaufmann.
- Quentin Lhoest, Albert Villanova del Moral, Yacine Jernite, Abhishek Thakur, Patrick von Platen, Suraj Patil, Julien Chaumond, Mariama Drame, Julien Plu, Lewis Tunstall, Joe Davison, Mario Šaško, Gunjan Chhablani, Bhavitvya Malik, Simon Brandeis, Teven Le Scao, Victor Sanh, Canwen Xu, Nicolas Patry, and 13 others. 2021. Datasets: A community library for natural language processing. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 175–184.
- Andrew McCallum and Kamal Nigam. 1998. A comparison of event models for naive Bayes text classification. In *AAAI-98 Workshop on Learning for Text Categorization*, pages 41–48.
- Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeffrey Dean. 2013. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems 26*, pages 3111–3119.
- George A. Miller. 1995. WordNet: A lexical database for English. *Communications of the ACM*, 38(11):39–41.
- Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. 2013. On the difficulty of training recurrent neural networks. In *Proceedings of the 30th International Conference on Machine Learning*, pages 1310–1318.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.
- Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1532–1543.
- Martin F. Porter. 1980. An algorithm for suffix stripping. *Program*, 14(3):130–137.
- Radim Rehurek and Petr Sojka. 2010. Software framework for topic modelling with large corpora. In *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*, pages 45–50.
- Gerard Salton and Christopher Buckley. 1988. Term-weighting approaches in automatic text retrieval. *Information Processing and Management*, 24(5):513–523.
- Mike Schuster and Kuldip K. Paliwal. 1997. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11):2673–2681.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pages 1715–1725.
- Karen Spärck Jones. 1972. A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1):11–21.
- Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958.

Model	Err.	Within	Cross	Within %
BERT-Base	1,928	770	1,158	39.9
LogReg	2,192	798	1,394	36.4
NB	2,151	802	1,349	37.3
Bi-LSTM	2,348	956	1,392	40.7
LSTM	2,382	944	1,438	39.6
Bi-GRU	2,608	999	1,609	38.3
GRU	2,684	1,016	1,668	37.9
RandomForest	2,634	908	1,726	34.5
BiSimpleRNN	3,861	1,809	2,052	46.9
SimpleRNN	4,421	1,950	2,471	44.1

Table 5: Error structure by model. Random guessing would put about 15% of errors inside the same label family; every model runs far above that.

Chi Sun, Xipeng Qiu, Yige Xu, and Xuanjing Huang. 2019. How to fine-tune BERT for text classification? In *Chinese Computational Linguistics: 18th China National Conference*, pages 194–206. Springer.

Lewis Tunstall, Nils Reimers, Unso Eun Seo Jo, Luke Bates, Daniel Korat, Moshe Wasserblat, and Oren Pereg. 2022. Efficient few-shot learning without prompts. *arXiv preprint arXiv:2209.11055*.

Alper Kursat Uysal and Serkan Gunal. 2014. The impact of preprocessing on text classification. *Information Processing and Management*, 50(1):104–112.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, pages 5998–6008.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, and 3 others. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45.

Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alex Smola, and Eduard Hovy. 2016. Hierarchical attention networks for document classification. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1480–1489.

Wenpeng Yin, Jamaal Hay, and Dan Roth. 2019. Benchmarking zero-shot text classification: Datasets, evaluation and entailment approach. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3914–3923.

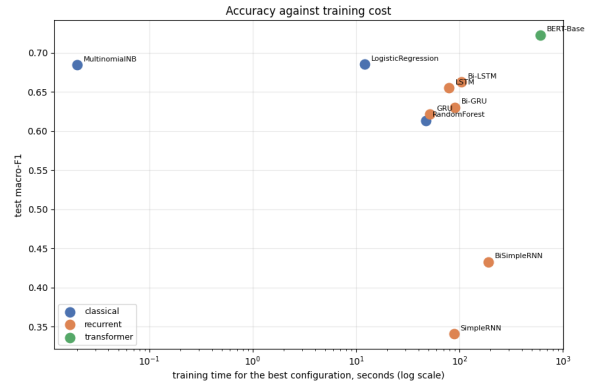


Figure 6: Test macro-F1 against training time for the best configuration of each model. The horizontal axis is log-scaled.

ID	Setting		
<i>Classical, on TF-IDF (text_clean)</i>			
LR-C1/C2/C3	$C = 1.0, 5.0, 10.0$		
NB-C1/C2/C3	$\alpha = 1.0, 0.1, 0.01$		
RF-C1	300 trees, sqrt, leaf 1		
RF-C2	600 trees, sqrt, leaf 2		
RF-C3	300 trees, 2% features, leaf 1		
<i>Recurrent, shared grid across all six models</i>			
	emb.	units / drop	trainable
S1	GloVe	64 / 0.3	no
S2	Word2Vec	64 / 0.3	no
S3	Word2Vec	128 / 0.5	yes
<i>Transformer, bert-base-uncased on raw text</i>			
	lr	batch	epochs
B1	2e-5	16	3
B2	3e-5	16	3
B3	5e-5	32	4

Table 6: All thirty required configurations. Each model varies the parameter that controls its own capacity.

A Error structure and training cost

B Configuration grid

C Full tuning table

Table 7 lists every one of the thirty required runs plus the six ablations, with validation scores and training times. All 36 rows carry the same dataset checksum.

D Per-class results for the best model

Table 8 gives the per-class test scores for BERT-Base at configuration B3.

E Class-family blocks

Figure 8 shows the three religion-related classes for the best model of each tier, the one block where the classical model is not beaten by the trans-

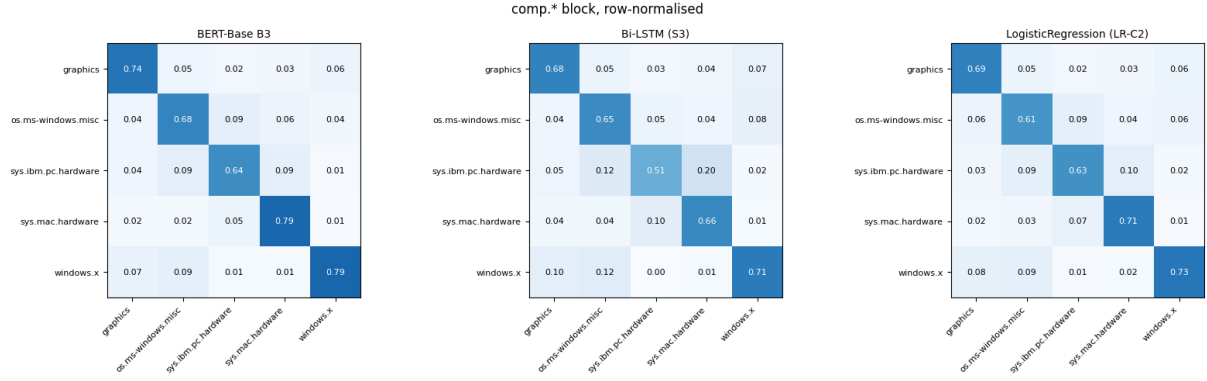


Figure 7: The `comp` block, row-normalised, for the best model of each tier. BERT holds an average diagonal of 0.727 against 0.673 for Logistic Regression and 0.641 for Bi-LSTM.

Model	Cfg	Val acc.	Val F1	Time
LogReg	LR-C1	0.7385	0.7241	10.4
LogReg	LR-C2	0.7477	0.7377	12.1
LogReg	LR-C3	0.7446	0.7353	11.0
LogReg ^a	LR-C2-1	0.7312	0.7218	15.9
NB	NB-C1	0.7110	0.6887	0.03
NB	NB-C2	0.7508	0.7348	0.02
NB	NB-C3	0.7404	0.7317	0.03
NB ^a	NB-C2-1	0.7336	0.7179	0.03
RF	RF-C1	0.6632	0.6459	63.2
RF	RF-C2	0.6767	0.6593	47.1
RF	RF-C3	0.6412	0.6238	97.7
RF ^a	RF-C2-1	0.6699	0.6518	82.7

SimpleRNN	S1	0.3319	0.2853	65.3
SimpleRNN	S2	0.3938	0.3351	91.8
SimpleRNN	S3	0.4317	0.3744	88.3
BiSimpleRNN	S1	0.4207	0.3880	132.6
BiSimpleRNN	S2	0.4501	0.4114	119.7
BiSimpleRNN	S3	0.4960	0.4640	190.0
LSTM	S1	0.6246	0.6120	76.1
LSTM	S2	0.6301	0.6094	78.6
LSTM	S3	0.7250	0.7122	78.9
LSTM ^a	ABL-clean	0.6320	0.6219	79.5
LSTM ^a	ABL-light	0.6099	0.5889	73.3
Bi-LSTM	S1	0.6522	0.6378	113.9
Bi-LSTM	S2	0.6320	0.6130	98.8
Bi-LSTM	S3	0.7397	0.7283	104.6
GRU	S1	0.6405	0.6287	71.6
GRU	S2	0.6314	0.6131	90.1
GRU	S3	0.6969	0.6888	51.7
Bi-GRU	S1	0.6399	0.6270	81.8
Bi-GRU	S2	0.6301	0.6104	65.1
Bi-GRU	S3	0.7116	0.7069	89.9

BERT-Base	B1	0.7679	0.7463	432.7
BERT-Base	B2	0.7777	0.7627	472.7
BERT-Base	B3	0.7832	0.7753	602.4
BERT-Base ^a	ABL-512	0.7887	0.7782	1113.9

Table 7: All 36 runs. Rows marked ^a are ablations and are not among the thirty required runs. Times are in seconds.

former. Average diagonals are 0.519 for Logistic Regression, 0.511 for BERT-Base and 0.469 for Bi-LSTM.

Class	P	R	F1	n
alt.atheism	0.497	0.479	0.488	309
comp.graphics	0.731	0.738	0.734	382
comp.os.ms-windows.misc	0.694	0.676	0.685	379
comp.sys.ibm.pc.hardware	0.672	0.641	0.656	384
comp.sys.mac.hardware	0.710	0.789	0.747	369
comp.windows.x	0.851	0.790	0.819	390
misc.forsale	0.846	0.857	0.851	377
rec.autos	0.827	0.758	0.791	367
rec.motorcycles	0.773	0.791	0.782	383
rec.sport.baseball	0.909	0.880	0.894	375
rec.sport.hockey	0.916	0.930	0.923	388
sci.crypt	0.779	0.775	0.777	377
sci.electronics	0.659	0.640	0.649	380
sci.med	0.834	0.876	0.855	379
sci.space	0.781	0.814	0.797	376
soc.religion.christian	0.763	0.737	0.750	384
talk.politics.guns	0.552	0.715	0.623	351
talk.politics.mideast	0.872	0.758	0.811	367
talk.politics.misc	0.539	0.483	0.510	302
talk.religion.misc	0.303	0.317	0.310	243
macro avg	0.725	0.722	0.723	7262
weighted avg	0.738	0.735	0.735	7262

Table 8: Per-class test results for BERT-Base (B3). The four weakest classes are the three religion-related groups and `talk.politics.misc`.

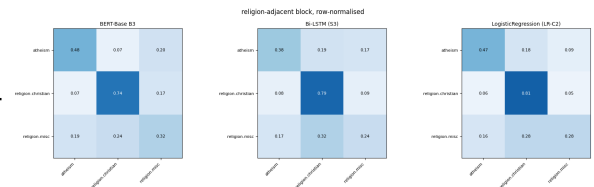


Figure 8: The religion-adjacent block, row-normalised, for the best model of each tier. Logistic Regression reaches an average diagonal of 0.519 against 0.511 for BERT and 0.469 for Bi-LSTM.